

AI Infra 工程师·高薪通关终极作战地图

模块一：掌握核心武器（语言与系统内功）

目标：具备写出高性能后端服务的能力，并拥有阅读底层框架源码的硬实力。

1. Go 语言（工程主力，云原生世界的绝对母语）

- **并发编程进阶**：不仅要会用 goroutine 和 channel，还要深入理解 GMP 调度模型（Goroutine、Machine、Processor 是如何协作的），掌握 sync.WaitGroup、sync.Mutex、sync.Once 以及原子操作（atomic）的使用场景。
- **高级语法与工程化**：精通 Context 上下文控制（如何在微服务链路中传递超时与取消信号）；掌握反射（reflect）与接口（interface）的底层原理；熟练使用 Go Modules 进行依赖管理；掌握单元测试（testing）与基准测试（benchmark）。
- **网络与微服务开发**：熟练使用 Gin 或 Echo 框架编写高性能 RESTful API；精通 gRPC 与 Protobuf 的定义与调用；理解 HTTP/2 与 TCP/IP 协议栈在 Go 中的实现。
- **性能调优**：学会使用 pprof 分析 CPU 火焰图与内存分配，能够定位并解决 Goroutine 泄漏、内存逃逸等实际问题。

2. C++（性能主力，通往底层优化的钥匙）

- **现代 C++ 核心**：熟练掌握 C++11/14/17 新特性，特别是智能指针（unique_ptr, shared_ptr）如何避免内存泄漏，以及右值引用与移动语义（Move Semantics）如何减少不必要的内存拷贝。
- **系统与内存管理**：深入理解指针操作、手动内存管理（new/delete）与栈/堆的区别；掌握多线程编程（std::thread, std::mutex, std::condition_variable）。
- **构建与编译**：学会使用 CMake 编写跨平台的构建脚本，理解编译、链接的基本过程。
- **源码阅读能力**：不需要成为 C++ 专家，但要能看懂 PyTorch 或 vLLM 中核心算子（如 Attention 计算、矩阵乘法 GEMM）的 C++/CUDA 实现逻辑。

3. Linux 系统内功（一切性能的基石）

- **核心原理**：深入理解 Linux 的进程/线程调度、虚拟内存管理、零拷贝（Zero-Copy）技术。
- **IO 模型**：彻底搞懂阻塞 IO、非阻塞 IO、IO 多路复用（select/poll/epoll）的区别，理解为什么 Nginx、Redis 和 Go 的网络库都依赖 epoll。
- **性能排查工具链**：熟练使用 top/htop 看负载，free 看内存，iostat 看磁盘 IO，netstat/ss 看网络连接；进阶掌握 perf 进行系统级性能采样，strace 追踪系统调用，tcpdump 抓包分析网络问题。

模块二：构建资源底座（云原生与 GPU 调度）

目标：能像管理水电一样，在大规模集群上自动化管理和调度昂贵的算力资源。

1. 容器化深度掌握

- **镜像工程**：精通 Dockerfile 的最佳实践（如多阶段构建减小镜像体积、利用构建缓存）；理解镜像的分层存储原理与 UnionFS 联合文件系统。
- **容器底层**：深入理解 Namespace（如何实现网络、PID、挂载点的隔离）与 Cgroups（如何实现 CPU、内存的资源限制）。
- **运行时**：了解 containerd 与 runc 的基本工作流程，知道 Docker 命令背后到底调用了什么。

2. Kubernetes (K8s) 核心（云原生时代的操作系统）

- **工作负载管理**：精通 Pod 的生命周期（Init 容器、健康检查 Liveness/Readiness Probe）；掌握 Deployment（无状态应用）、StatefulSet（有状态应用，如数据库）、DaemonSet（节点守护进程）的适用场景与配置。
- **网络与服务发现**：深入理解 K8s 网络模型（CNI 插件，如 Calico/Flannel）；掌握 Service 的四种类型（ClusterIP, NodePort, LoadBalancer, ExternalName）及 kube-proxy 的 iptables/IPVS 转发模式；学会配置 Ingress 进行七层路由。
- **配置与存储**：熟练使用 ConfigMap 和 Secret 管理配置与敏感信息；彻底搞懂 PV（持久卷）、PVC（持久卷声明）、StorageClass（存储类）的动态供给机制。
- **运维与排障**：学会使用 kubectl 常用命令排查 Pod 启动失败（CrashLoopBackOff, ImagePullBackOff 等）的原因；掌握 Helm 的 Chart 编写与模板渲染。

3. GPU 调度（AI Infra 的核心差异化竞争力）

- **容器调用 GPU**：掌握 NVIDIA Container Toolkit（原 nvidia-docker）的安装与配置，理解它是如何将宿主机的 GPU 驱动与设备文件映射到容器内的。
- **K8s GPU 资源管理**：深入理解 K8s 的 Device Plugin 机制，学会部署 NVIDIA Device Plugin，让 K8s 能够识别节点上的 GPU 数量与健康状态。
- **高级调度策略**：了解如何配置 GPU 的资源限制（limits/requests）；学习 Volcano 调度器，理解 Gang Scheduling（帮派调度，即多机多卡训练时，所有任务要么同时启动，要么同时等待）和 Binpacking（装箱策略，尽量把任务塞满少数节点以节省资源）。

模块三：攻克存储与数据（分布式存储与向量检索）

目标：解决海量模型文件与业务数据的持久化存储、高并发读写及高效检索问题。

1. 分布式存储（数据的大本营）

- **对象存储实战**：精通 MinIO 或 Ceph (RGW) 的集群搭建、扩容与运维；理解对象存储的扁平化结构、元数据管理以及纠删码（Erasure Coding）数据保护原理。
- **K8s 存储集成 (CSI)**：深入理解 CSI（容器存储接口）标准，学会编写或配置 CSI 插件，实现在 K8s 中动态创建云盘或对象存储桶并挂载到 Pod。
- **高性能存储网络**：了解 NVMe 协议相比传统 SSD 的优势；理解 RDMA（远程直接内存访问）与 RoCE 网络如何绕过 CPU 实现节点间的超低延迟数据传输（这是千卡集群训练的必备技术）。

2. 向量数据库（AI 时代的“记忆中枢”）

- **检索原理**：理解高维向量检索的基本概念，搞懂暴力搜索与近似最近邻搜索（ANN）的区别；深入理解 HNSW（分层可导航小世界）图索引算法的基本原理。
- **引擎实战**：掌握 Milvus 或 Faiss 的架构与部署；学会使用 Python/Go SDK 进行数据的增删改查；掌握如何调整索引参数（如 nlist, nprobe）来平衡检索精度与延迟。
- **业务落地**：理解 Embedding 模型如何将文本/图片转化为向量；掌握 RAG（检索增强生成）架构中“切片 -> 向量化 -> 存储 -> 检索 -> 拼接 Prompt”的全流程。

3. 传统数据库与缓存（业务数据的基石）

- **关系型数据库**：掌握 MySQL/PostgreSQL 的事务隔离级别、索引优化（B+树原理）与主从复制架构。
- **分布式缓存**：精通 Redis 的核心数据结构（String, Hash, List, Set, Zset）及其底层实现；掌握 Redis 集群（Cluster）与哨兵（Sentinel）模式的高可用部署；理解缓存穿透、缓存击穿与缓存雪崩的解决方案。

模块四：登顶性能优化（推理服务与缓存加速）

目标：解决巨型程序（大模型）运行“慢”和“贵”的痛点，这是目前大厂开出高薪的核心领域。

1. 高性能服务部署与加速

- **推理引擎实战**：精通 vLLM、Triton Inference Server 或 SGLang 的部署；掌握其核心配置参数（如最大并发请求数、最大上下文长度）。
- **模型优化技术**：理解模型量化（Quantization）原理，学会使用 AWQ、GPTQ 等技术将模型从 FP16 压缩到 INT8 或 INT4，大幅降低显存占用；了解模型编译技术（如 TorchDynamo, TVM）的基本概念。
- **服务化架构**：掌握如何将推理服务封装为标准的 RESTful/gRPC API；理解动态批处理（Dynamic Batching）如何通过合并多个请求来极大提升 GPU 的吞吐量。

2. KVCache 深度优化（显存管理的艺术）

- **显存占用分析**：深入理解 Transformer 在推理（Decode）阶段，KVCache 是如何随着生成长度线性增长的，以及它为什么是显存占用的“大头”。
- **PagedAttention 原理**：彻底搞懂 vLLM 的核心技术——PagedAttention。理解它如何借鉴操作系统的虚拟内存分页思想，将连续的 KVCache 切分成非连续的块（Block），从而彻底解决显存碎片化问题。
- **前缀缓存（Prefix Caching）**：理解在多轮对话或 System Prompt 相同的场景下，如何通过复用已有的 KVCache 块，避免重复计算，将首字延迟（TTFT）降低一个数量级。

3. 全链路监控与可观测性

- **监控体系搭建**：掌握 Prometheus（数据采集）+ Grafana（可视化展示）的部署与配置。

- **核心指标监控**：学会监控 K8s 集群资源（CPU/内存/GPU利用率）、推理服务指标（QPS、P99 延迟、Token 生成速度、KVCache 命中率）。
- **日志与追踪**：了解 ELK（Elasticsearch, Logstash, Kibana）日志栈或 Loki 的基本使用；理解分布式追踪（如 OpenTelemetry）在微服务链路排查中的作用。

终极实战：三份能写进简历的“高薪项目清单”

项目一：基于 K8s 的分布式模型仓库（夯实存储与调度）

- **项目描述**：在 K8s 集群中搭建一套 MinIO 分布式对象存储，并开发一个 Go 语言的后端服务，实现超大文件（模拟大模型文件）的上传、下载、分片传输和版本管理。
- **涵盖技术**：Go、K8s (Deployment/Service/PVC)、MinIO、Docker。
- **简历亮点**：解决了超大文件在集群内的持久化存储与高效分发问题，实现了基于 K8s 的存算分离架构。

项目二：企业级 RAG 知识库问答平台（打通向量数据库）

- **项目描述**：搭建一套完整的检索增强生成（RAG）系统。用户上传技术文档，系统自动向量化存入 Milvus；用户提问时，先从 Milvus 检索相关知识，再交给后端服务生成答案。
- **涵盖技术**：向量数据库 (Milvus/Faiss)、Go/Python 后端、大模型 API 调用。
- **简历亮点**：实现了非结构化数据的分布式存储与高维向量检索，落地了当前最火的业务架构。

项目三：高性能推理网关与缓存优化（冲刺高薪核心）

- **项目描述**：基于 vLLM 部署高性能推理服务，并开发一个 Go 语言的高并发 API 网关。在网关层实现“前缀缓存（Prefix Caching）”策略，当多个用户提问相似内容时，复用缓存，降低首字延迟。
- **涵盖技术**：vLLM、KVCache 原理、PagedAttention、K8s HPA (自动扩缩容)、Go 高并发网关。
- **简历亮点**：深入服务底层，通过缓存优化与资源管理，将服务的并发吞吐量提升了 X%，首字延迟降低了 Y%。